

FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA

Felipe Monte, RM562019, 2TDSPG
Gabriel Neris Losano, RM564093, 2TDSPG
Joao Vitor Biribilli, RM565594, 2TDSPG
Pedro de Matos, RM564184, 2TDSPG
Pietro Paranhos Wilhelm, 561378, 2TDSPG

PetPulse

São Paulo
2026

Link do repositório: <https://github.com/Fiap-2tdspg-Challenge/PetPulse-Api>

Cronograma de desenvolvimento

1. Levantamento do problema

Responsável: Joao Vitor Biribilli

Período previsto: 01/05/2026

Realizado em: 01/05/2026

Status: Concluído

Nesta etapa, o grupo analisou a proposta do projeto e identificou o problema que a aplicação PetPulse busca resolver: a dificuldade dos tutores em organizar e acompanhar informações importantes sobre a saúde e rotina dos seus pets.

2. Definição da solução

Responsável: Joao Vitor Biribilli

Período previsto: 02/05/2026

Realizado em: 02/05/2026

Status: Concluído

Foi definida a proposta da aplicação PetPulse API, uma solução voltada para o cadastro de tutores, pets, histórico clínico, dispositivos IoT e alertas inteligentes relacionados à saúde animal.

3. Modelagem das classes de domínio

Responsável: Joao Vitor Biribilli

Período previsto: 03/05/2026 a 04/05/2026

Realizado em: 03/05/2026 a 04/05/2026

Status: Concluído

Foram definidas as principais classes de domínio da aplicação: **Tutor**, **Pet**, **ClinicalHistory**, **IoTDevice** e **SmartAlert**, além dos enums necessários para representar tipos, status e classificações do sistema.

4. Criação do Diagrama Entidade-Relacionamento

Responsável: Joao Vitor Biribilli

Período previsto: 05/05/2026

Realizado em: 05/05/2026

Status: Concluído

Nesta etapa, foi elaborado o DER do projeto, representando as tabelas do banco de dados, suas chaves primárias, chaves estrangeiras e os relacionamentos entre as entidades.

5. Criação do Diagrama de Classes das Entidades

Responsável: Joao Vitor Biribilli

Período previsto: 06/05/2026

Realizado em: 06/05/2026

Status: Concluído

Foi desenvolvido o Diagrama de Classes das Entidades, mantendo coerência com o DER e representando os atributos, enums e relacionamentos entre as classes de domínio.

6. Configuração do projeto Spring Boot

Responsável: Joao Vitor Biribilli

Período previsto: 07/05/2026 a 08/05/2026

Realizado em: 07/05/2026 a 08/05/2026

Status: Concluído

O projeto foi configurado com Spring Boot, estrutura de pacotes, dependências necessárias, configuração do banco de dados e organização inicial das camadas da aplicação.

7. Implementação das entidades JPA

Responsável: Joao Vitor Biribilli

Período previsto: 09/05/2026 a 10/05/2026

Realizado em: 09/05/2026 a 10/05/2026

Status: Concluído

As entidades foram implementadas com anotações JPA, incluindo mapeamento de tabelas, chaves primárias, relacionamentos e enums. Também foram utilizados métodos automáticos como `@PrePersist` para preencher datas de criação ou geração.

8. Criação dos Repositories

Responsável: Joao Vitor Biribilli

Período previsto: 11/05/2026

Realizado em: 11/05/2026

Status: Concluído

Foram criadas as interfaces Repository responsáveis pela comunicação com o banco de dados, utilizando Spring Data JPA para facilitar as operações de cadastro, consulta, atualização e exclusão.

9. Criação dos Services

Responsável: Joao Vitor Biribilli

Período previsto: 12/05/2026 a 13/05/2026

Realizado em: 12/05/2026 a 13/05/2026

Status: Concluído

Foram desenvolvidas as classes de serviço, responsáveis por concentrar as regras de negócio, realizar validações e fazer a comunicação entre os controllers e os repositories.

10. Criação dos Controllers

Responsável: Joao Vitor Biribilli

Período previsto: 14/05/2026 a 15/05/2026

Realizado em: 14/05/2026 a 15/05/2026

Status: Concluído

Foram implementados os controllers da API, responsáveis por disponibilizar os endpoints REST para gerenciamento de tutores, pets, históricos clínicos, dispositivos IoT e alertas inteligentes.

11. Criação dos DTOs

Responsável: Joao Vitor Biribilli

Período previsto: 16/05/2026

Realizado em: 16/05/2026

Status: Concluído

Foram criados os DTOs de entrada e saída de dados, com o objetivo de organizar melhor as informações recebidas e retornadas pela API, evitando a exposição direta das entidades.

12. Configuração do Swagger

Responsável: Joao Vitor Biribilli

Período previsto: 17/05/2026

Realizado em: 17/05/2026

Status: Concluído

Foi configurada a documentação da API utilizando Swagger/OpenAPI, permitindo visualizar, testar e documentar os endpoints disponíveis na aplicação.

13. Testes da API

Responsável: Joao Vitor Biribilli

Período previsto: 18/05/2026

Realizado em: 18/05/2026

Status: Concluído

Os endpoints foram testados utilizando ferramentas como Swagger, Postman ou Insomnia, verificando se as operações principais estavam funcionando corretamente.

14. Revisão final da entrega

Responsável: Joao Vitor Biribilli

Período previsto: 19/05/2026

Realizado em: 19/05/2026

Status: Concluído

O grupo realizou a revisão final do código, dos diagramas, da documentação e da organização do repositório, garantindo que os artefatos estivessem coerentes com os requisitos da Sprint.

15. Entrega da 1ª Sprint

Responsável: Todos os integrantes

Período previsto: 20/05/2026

Realizado em: 20/05/2026

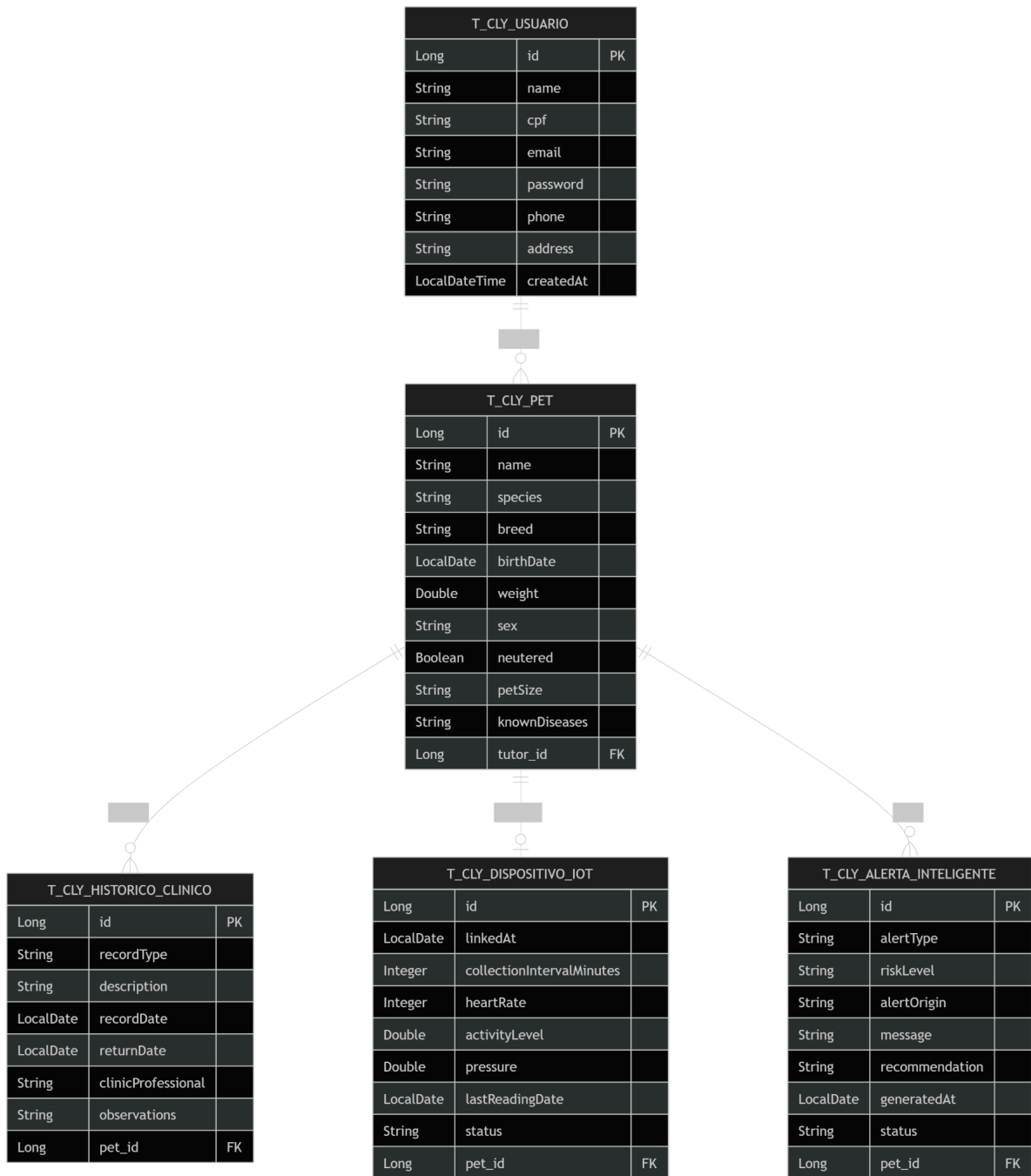
Status: Concluído

A entrega final da 1ª Sprint foi organizada e enviada dentro do prazo, contendo os artefatos solicitados, como arquitetura da aplicação, definição das classes de domínio, DER, Diagrama de Classes das Entidades e cronograma de desenvolvimento.

Diagrama de Classes das Entidades



DER — Diagrama Entidade-Relacionamento



5. Explicação dos relacionamentos e constraints

O relacionamento entre **Tutor** e **Pet** é de um para muitos, pois um tutor pode cadastrar vários pets, enquanto cada pet pertence a apenas um tutor.

O relacionamento entre **Pet** e **ClinicalHistory** também é de um para muitos, pois cada pet pode possuir diversos registros clínicos, como vacinas, exames, consultas e medicamentos.

O relacionamento entre **Pet** e **IoTDevice** é de um para um, pois a modelagem considera que um dispositivo IoT está vinculado a um único pet.

O relacionamento entre **Pet** e **SmartAlert** é de um para muitos, já que um pet pode gerar vários alertas inteligentes ao longo do tempo.

As chaves primárias das entidades são representadas pelo atributo **id**, gerado automaticamente com **GenerationType.IDENTITY**. As chaves estrangeiras aparecem nas tabelas relacionadas, como **tutor_id** em **T_CLY_PET** e **pet_id** nas tabelas **T_CLY_HISTORICO_CLINICO**, **T_CLY_DISPOSITIVO_IOT** e **T_CLY_ALERTA_INTELIGENTE**. Os atributos enum são armazenados como texto por meio de **@Enumerated(EnumType.STRING)**, o que melhora a legibilidade dos dados no banco.